VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF APPLIED SCIENCE

CALCULUS 1 (MT1003)

Class: CC02

ASSIGNMENT REPORT

Topic 8: Crack Detection 1

Instructor:	Assoc. Prof.Phan Thành An	
Students:	Âu Lê Quang Trí	2453300
	Dương Quốc Huy	2452371
	Ngô Phúc Hiển	2452339
	Tôn Thất Duy Hiển	2453460

Ho Chi Minh City, December 10th, 2024

Contents

Chapter 1: Introduction **3**

1 What is Crack? **3**

2 Important of crack detection **3**

3 Crack detection using Opencv **3**

Chapter 2: OpenCV Overview **4**

1 Definition of OpenCV **4**

2 Features of OpenCV Library[12] **4**

3 Key advantages of OpenCV[12] **4**

Chapter 3: Picture processing **5**

Chapter 4: Calculus in functions **7**

1 Grayscale [4] **7**

2 Bilateral Filter **8**
2.1 The syntax for the bilateral filter in OpenCV [5] 8
2.2 The formula of the bilateral filter[6] 8
2.3 Solving an example manually 9

3 Canny **11**
3.1 Noise Reduction[8] 11
3.2 Gradient Calculation[8] 12
3.3 Non-Maximum Suppression[8] 13
3.4 Double Thresholding[8] 15
3.5 Edge Tracking by Hysteresis[8] 16
3.6 Output of Canny edge detection[9] 17

4 MorphologyEx **19**
4.1 Function Syntax 19
4.2 Dilation Operation[3] 20
4.3 Erosion Operation[3] 21

5 Contours Detection **22**
5.1 Function syntax[13] 22
5.2 Mathematical Definition[15] 22
5.3 Algorithm of contours detection 23
5.4 Results[13] 23

Chapter 5: Final product **24**

1 The source code of crack detection program **24**

Chapter 6: Conclusion **26**

References **27**

Chapter 1: Introduction

1 What is Crack?

The context of infrastructure and structural engineering, a crack is a physical separation or fracture within a material, such as concrete, metal, or masonry, that compromises the material's integrity.[2]
The causes of cracks are due to external forces and changes in environmental factors, such as moisture, temperature, elastic deformation, and expansion. These usually happen after natural disasters like storms, floods, and earthquakes[1].

2 Important of crack detection

Cracks can damage the inner finishes, increasing maintenance costs or causing corrosion of the reinforcement. This has a long-term negative impact on the structure's stability and may eventually lead to the corruption of the entire structure. Active cracks are a major problem that need to be detected immediately, as they are structurally unsafe and can directly endanger human life.[2]

3 Crack detection using Opencv

Digital images of structures (e.g., bridges or concrete walls) are processed using algorithms to automatically detect cracks. Techniques such as edge detection, texture analysis, and machine learning can improve crack identification.[10]

- Advantages: Automated, fast, and able to cover large surface areas.
- Applications: Civil infrastructure monitoring and automated inspections in manufacturing.

Chapter 2: OpenCV Overview

1 Definition of OpenCV

OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library developed by Intel. It provides a common infrastructure for applications related to computer vision and its associated fields. It is used to speed up the use of real-time machine recognition of images, objects, and video processing applications.[11]

2 Features of OpenCV Library[12]

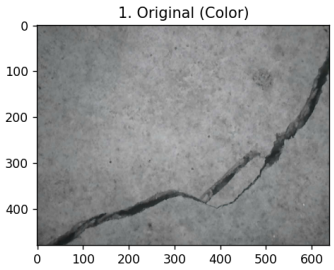
- Read and write images, videos.
- Image processing such as filtering and transformation .
- Feature detection.
- Video or image object detection such as human body parts, cars, signage, etc.
- Video analysis.

3 Key advantages of OpenCV[12]

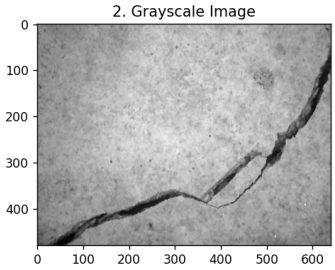
- It supports a wide range of programming languages which include C++, Java, Python, etc.
- OpenCV is presently the largest computer vision library with more than 2500 algorithms and has evolved tremendously over the years.
- OpenCV provides an accessible and easy-to-use computer vision infrastructure that helps people build computer vision applications quickly.

Chapter 3: Picture processing [7]

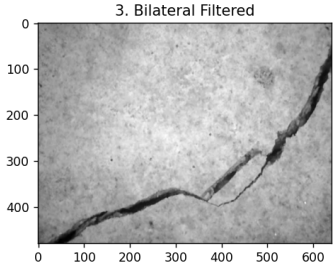
1. Input image:



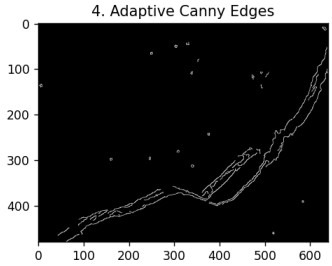
2. Convert to grayscale image:



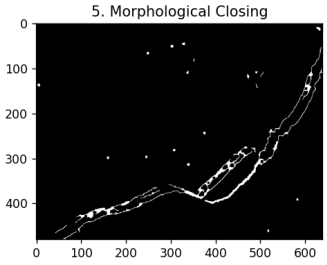
3. Using Bilateral Filter to reduce noise:



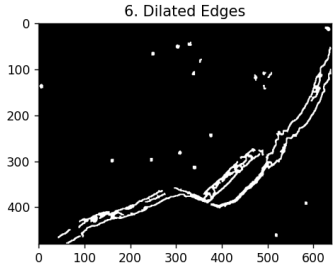
4. Using Adaptive Canny Thresholds for edge detection:



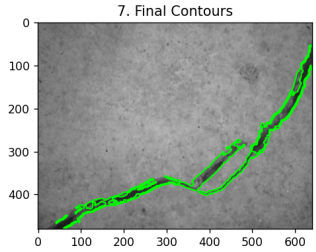
5. Morphological Operations for better edge continuity:



6. Apply dilation to enhance the cracks further and detect the contours:



7. Drawing Contours on the original image for better visualization:



Chapter 4: Calculus in functions

1 Grayscale [4]

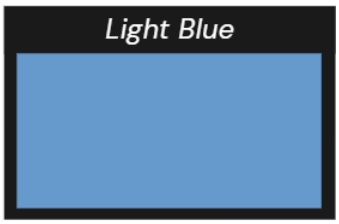
Grayscaling is the process of transforming an image from its original color space into a grayscale. This involves converting the image into varying shades of gray, ranging from pure black to pure white.
Instead of 3 channels RGB as default, it always converts the image to the single-channel grayscale image by applying grayscale conversion.

The formula used is:

$$Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

where R, G, and B are the red, green, and blue components of a pixel, respectively, and Y is the calculated luminance (grayscale intensity).
Let's take an example RGB pixel:

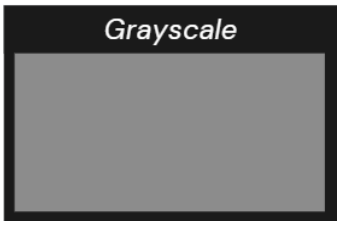
- $R = 100$
- $G = 150$
- $B = 200$



The grayscale value (Y) for this pixel would be calculated as:

$$Y = (0.299 \times 100) + (0.587 \times 150) + (0.114 \times 200) = 140.75$$

So, the resulting grayscale intensity value would be approximately 141 (rounded to the nearest integer).



2 Bilateral Filter

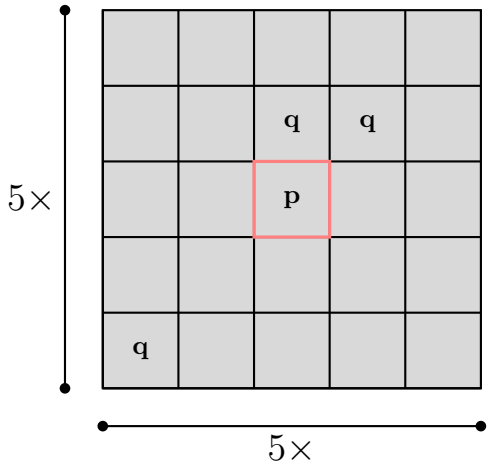
The bilateral filter is an advanced image-smoothing technique that reduces noise while preserving the edges of the image. It takes into account both the spatial distance and the color intensity differences when averaging pixels. This function of intensity makes sure that only those nearby pixels with similar intensities to the central pixel are considered for blurring. So it preserves the edges since pixels at the edges will have large intensity variation.[5]

2.1 The syntax for the bilateral filter in OpenCV [5]

```
cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace)
```

This function takes four arguments:

- **src:** Represents the source (input image) for this operation (e.g., `img`).
- **d:** Diameter of each pixel neighborhood that is used during filtering.
  - Larger `d` values help reduce noise but can sometimes over-smooth fine details.
  - Smaller `d` values preserve more detail but may not reduce noise as effectively.
  - Example: $d = 5$ S



- **sigmaColor:** Filter sigma in the color space. A larger value of this parameter means that farther colors within the pixel neighborhood will be mixed together, resulting in larger areas of semi-equal color.
- **sigmaSpace:** Filter sigma in the coordinate space. A larger value of this parameter means that farther pixels will influence each other as long as their colors are close enough.

2.2 The formula of the bilateral filter[6]

$$New[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|)\, G_{\sigma_r}(|I_p - I_q|)\, I_q$$

Explanation:

- p: Symbolizes the pixel at the center.
- q: Symbolizes the pixels surrounding the central pixel.
- I: The intensity of the pixel that represents its brightness.
- $New[I]_p$: the result intensity value for the central pixel after applying the filter.
- $\sum_{q \in S}$: This summation goes over all pixels q within a neighborhood S around pixel P.
- $G\sigma_s(\|p-q\|)$: a function with a variable ($\|p-q\|$), calculates weight based on the spatial (geometric) distance between two pixels[7] .
  - It is calculated as:
$$G\sigma_s(\|p-q\|) = exp\left(-\frac{\|p-q\|^2}{2\sigma_s^2}\right)$$
  - $\|p-q\|$ is the distance between p and q, which is calculated as:
$$\|p-q\| = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}$$
  - Sigma space controls the degree of spatial filtering: the larger the Sigma space, the more pixels farther away from p will influence its new value.
- $G\sigma_r(|I_p - I_q|)$: a function calculates weight based on the intensity difference between two pixels[7].
  - It is calculated as:
$$G\sigma_r(|I_p - I_q|) = exp\left(-\frac{|I_p - I_q|^2}{2\sigma_r^2}\right)$$
  - $|I_p - I_q|$ is the intensity difference between pixels p and q.
  - Sigma color ensures that pixels with similar intensities to I_p have a stronger influence on the final value of I_p, while pixels with different intensities have a reduced influence. This helps preserve edges.
- W_p: normalization factor, defined as the sum of all weights for the neighborhood around p[7].
  - It is calculated as:
$$W_p = \sum_{q \in S} G\sigma_s\left(\|p-q\|\right).G\sigma_r(|I_p - I_q|)$$
  - This ensures that the resulting intensity remains within the correct range by making the filtered value an average rather than an unnormalized sum.

2.3 Solving an example manually

The target pixel p is at the center $(x_p, y_p) = (10, 10)$
We have a neighboring pixel q_1, say $(x_q, y_q) = (9, 11)$

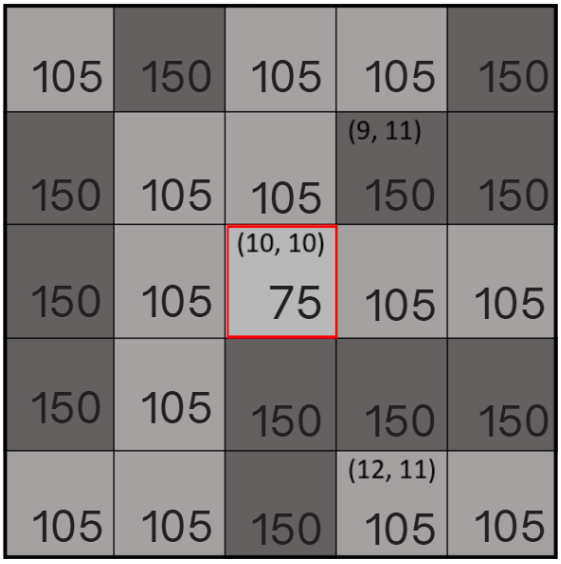


Figure 1: 5x5 square of pixels

- Spatial Gaussian function:

$$Distance : \|p-q\| = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = \sqrt{(10-9)^2 + (10-11)^2} \approx 1.41$$

With $\sigma_s = 3$

$$G_{\sigma_s(1.41)} = exp\left(-\frac{\|p-q\|^2}{2 \times 3^2}\right) = exp\left(-\frac{1.41^2}{18}\right) \approx 0.925 = 92.5\%$$

- Range Gaussian function:

$$Intensity : I_p - I_q = 150 - 75 = 75$$

With $\sigma_r = 25$

$$G\sigma_r(75) = exp\left(-\frac{|I_p - I_q|^2}{2\sigma_r^2}\right) = exp(-\frac{75^2}{2 \times 25^2}) = exp\left(-4.5\right) \approx 0.0111 = 1.11\%$$

So, we have:

$$G_{\sigma_s(1.41)}.G\sigma_r(75) = 1.11\% \times 92.5\% = 1.03\%$$
$$G_{\sigma_s(1.41)}.G\sigma_r(75).I_q = 1.03\% \times 150 = 1.545$$

Perform the same actions for q_2 with $(x_q, y_q) = (12, 11)$, we have:

$$G_{\sigma_s(2.24)}.G\sigma_r(30) = 75.7\% \times 48.8\% = 36.9\%$$

$$G_{\sigma_s(2.24)}.G\sigma_r(30).I_q = 36.9\% \times 105 = 38.745$$

Perform the same actions for q_3, q_4,... with $q \in S$, at last:

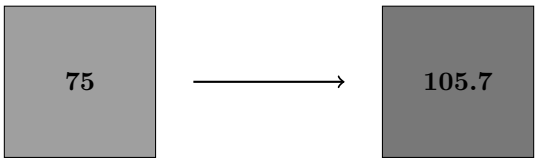
$$\sum_{q \in S} \left[G\sigma_s\left(\|p-q\|\right).G\sigma_r\left(|I_p - I_q|\right).I_q\right] = 1.545 + 38.745 + ... = 549.3$$

$$W_p = \sum_{q \in S} G\sigma_s\left(\|p-q\|\right).G\sigma_r(|I_p - I_q|) = 1.03\% + 36.9\% + ... = 5.1971$$

Result:

$$New[I]_p = \frac{549.345}{5.1971} = 105.7$$

The intensity of the central pixel changes from 75 to 105.7 after filtering:



3 Canny

Canny Edge Detection is a multi-step image processing technique used to identify edges in an image. Developed by John F. Canny in 1986, it is one of the most widely used edge detection algorithms due to its optimal performance in identifying edges without excessive noise.[9]

3.1 Noise Reduction[8]

Noise reduction in Canny edge detection is typically achieved using a Gaussian filter. The Gaussian filter smooths the image, reducing high-frequency noise while preserving significant features, such as edges. The mathematical representation of a Gaussian filter is as follows.

$$G(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

where:

- x, y are the pixel coordinates,
- σ is the standard deviation of the Gaussian kernel, controlling the degree of smoothing.

The Gaussian kernel is applied to the image by convolution, producing a smoothed version that is less susceptible to noise. The kernel is applied to the image through convolution, smoothing it and reducing the effects of noise. The size of the kernel (e.g., 3×3, 5×5) and the choice of σ determine the amount of smoothing.

Example

For a 5×5 Gaussian kernel with $\sigma = 2.0$, the values are symmetrically distributed around the center. Below is an example of such a kernel (normalized so that the sum of all elements is 1):

$$G = \frac{1}{159}\begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}$$

Here, each value in the kernel represents the weight applied to the corresponding pixel in the image during convolution.
Consider the following 5×5 image patch:

$$I = \begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 20 & 20 & 20 & 10 \\ 10 & 20 & 30 & 20 & 10 \\ 10 & 20 & 20 & 20 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{bmatrix}$$

The convolution operation applies the Gaussian kernel to the image patch by taking the dot product of the kernel and the corresponding overlapping patch of the image. For the center pixel, the convolution result is calculated as:

$$R(3,3) = \sum_{i=-2}^{2} \sum_{j=-2}^{2} G(i,j) \cdot I(3+i, 3+j)$$

Substituting values:

$$R(3,3) = \frac{1}{159} \left(2 \cdot 10 + 4 \cdot 10 + 5 \cdot 10 + 4 \cdot 10 + 2 \cdot 10 + \cdots + 15 \cdot 30\right)$$

Computing further, this gives:

$$R(3,3) = \frac{1}{159} \cdot 740 = 4.65$$

Similarly, the convolution is repeated for other pixels to produce the smoothed image.
The convolved image with Gaussian smoothing is as follows (rounded for simplicity):

$$\begin{bmatrix} 4 & 6 & 6 & 6 & 4 \\ 6 & 10 & 12 & 10 & 6 \\ 6 & 12 & 15 & 12 & 6 \\ 6 & 10 & 12 & 10 & 6 \\ 4 & 6 & 6 & 6 & 4 \end{bmatrix}$$

3.2 Gradient Calculation[8]

The gradient is calculated using the partial derivatives of the image intensity values in the x- and y-directions. This is achieved by convolving the smoothed image with Sobel filters.

Using Sobel Operator

The Sobel filters are used to approximate the partial derivatives:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Let $I(x, y)$ be the smoothed image. The gradient components in the x- and y-directions are computed as:

$$I_x = I * G_x, \quad I_y = I * G_y$$

where $*$ denotes the convolution operation.

Gradient Magnitude and Direction

The magnitude of the gradient at each pixel is given by:

$$G = \sqrt{I_x^2 + I_y^2}$$

This represents the strength of the edge at a given location.
The direction of the gradient is calculated as:

$$\theta = \arctan\left(\frac{I_y}{I_x}\right)$$

Here, θ gives the angle of the edge relative to the horizontal axis.

Example

Consider a simple 3×3 image patch:

$$I = \begin{bmatrix} 10 & 10 & 10 \\ 20 & 20 & 20 \\ 30 & 30 & 30 \end{bmatrix}$$

Convolving I with G_x:

$$I_x = \begin{bmatrix} -40 & 0 & 40 \\ -40 & 0 & 40 \\ -40 & 0 & 40 \end{bmatrix}$$

Convolving I with G_y:

$$I_y = \begin{bmatrix} -20 & -20 & -20 \\ 0 & 0 & 0 \\ 20 & 20 & 20 \end{bmatrix}$$

The gradient magnitude at the center pixel is:

$$G = \sqrt{I_x^2 + I_y^2} = \sqrt{0^2 + 0^2} = 0$$

The gradient direction at the center pixel is undefined because both I_x and I_y are zero.

Conclusion

Gradient calculation is a fundamental step in the Canny edge detection process. It identifies the strength and direction of edges, providing critical information for further steps like non-maximum suppression and edge tracking by hysteresis.

3.3 Non-Maximum Suppression[8]

Non-Maximum Suppression (NMS) is a crucial step in the Canny edge detection algorithm. It is applied to the gradient magnitude image to thin the edges, ensuring that only the most significant edges are retained. This step eliminates spurious responses, resulting in sharp and precise edges.

Purpose of Non-Maximum Suppression

The gradient magnitude calculated in the previous step often results in thick edges. NMS reduces these thick edges to one-pixel-wide lines by preserving pixels that are local maxima in the gradient direction.

Process of Non-Maximum Suppression

The process involves the following steps:

1. For each pixel in the gradient magnitude image, determine the direction of the gradient using the gradient angle θ.
2. Compare the pixel's gradient magnitude with its neighbors along the gradient direction.
3. Retain the pixel only if its magnitude is greater than those of its neighbors; otherwise, set it to zero.

Gradient Direction Quantization The gradient direction θ is quantized into four primary directions for simplicity:

- 0° (horizontal)
- 45° (diagonal rising)
- 90° (vertical)
- 135° (diagonal falling)

Mathematical Representation Let $G(x, y)$ be the gradient magnitude at pixel (x, y), and $\theta(x, y)$ be the gradient direction. The pixel (x, y) is compared with its neighbors along the quantized gradient direction:

- If $\theta \approx 0^\circ$, compare $G(x, y)$ with $G(x-1, y)$ and $G(x+1, y)$.
- If $\theta \approx 45^\circ$, compare $G(x, y)$ with $G(x-1, y+1)$ and $G(x+1, y-1)$.
- If $\theta \approx 90^\circ$, compare $G(x, y)$ with $G(x, y-1)$ and $G(x, y+1)$.
- If $\theta \approx 135^\circ$, compare $G(x, y)$ with $G(x-1, y-1)$ and $G(x+1, y+1)$.

If $G(x, y)$ is not the maximum among these, it is suppressed (set to zero).

Example

Consider a small 5×5 gradient magnitude image:

$$G = \begin{bmatrix} 0 & 10 & 20 & 10 & 0 \\ 10 & 30 & 40 & 30 & 10 \\ 20 & 40 & 50 & 40 & 20 \\ 10 & 30 & 40 & 30 & 10 \\ 0 & 10 & 20 & 10 & 0 \end{bmatrix}$$

Assume the quantized gradient direction for the center pixel (50) is 90°. Non-Maximum Suppression compares it with the vertical neighbors:

$$40 \quad \text{(above)} \quad \text{and} \quad 40 \quad \text{(below)}.$$

Since 50 is the maximum, it is retained. For other pixels in the image, a similar process is applied, resulting in a thinned edge image.

Conclusion

Non-Maximum Suppression is an essential step in the Canny edge detection algorithm, ensuring that only the most significant edges are retained. By thinning edges, it prepares the image for the final steps of edge tracking and hysteresis thresholding.

3.4 Double Thresholding[8]

Double Thresholding is a crucial step in the Canny edge detection algorithm. It classifies edges based on their gradient magnitudes into strong, weak, or non-edges, allowing the algorithm to identify and retain meaningful edges while discarding noise and irrelevant edges.

Purpose of Double Thresholding

Gradient magnitudes calculated during the earlier stages of the Canny process vary in strength. Double Thresholding ensures that:

- Strong edges (above a high threshold) are retained.
- Weak edges (between high and low thresholds) are retained only if they are connected to strong edges.
- Non-edges (below the low threshold) are suppressed.

Process of Double Thresholding

The Double Thresholding process involves:

1. Setting two thresholds: T_{high} and T_{low}, where $T_{\text{low}} < T_{\text{high}}$.
2. Classifying each pixel in the gradient magnitude image:
   - If the gradient magnitude is $\geq T_{\text{high}}$, mark it as a strong edge.
   - If $T_{\text{low}} \leq$ gradient magnitude $< T_{\text{high}}$, mark it as a weak edge.
   - If the gradient magnitude is $< T_{\text{low}}$, suppress the pixel (non-edge).

Example

Consider a 5×5 gradient magnitude image:

$$G = \begin{bmatrix} 0 & 20 & 30 & 10 & 0 \\ 10 & 50 & 80 & 50 & 10 \\ 20 & 70 & 100 & 70 & 20 \\ 10 & 50 & 80 & 50 & 10 \\ 0 & 20 & 30 & 10 & 0 \end{bmatrix}$$

Let $T_{\text{high}} = 75$ and $T_{\text{low}} = 25$. Applying Double Thresholding:

- Strong edges: $G(x, y) \geq 75$:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 80 & 0 & 0 \\ 0 & 0 & 100 & 0 & 0 \\ 0 & 0 & 80 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- Weak edges: $25 \leq G(x, y) < 75$:

$$\begin{bmatrix} 0 & 20 & 30 & 10 & 0 \\ 10 & 50 & 0 & 50 & 10 \\ 20 & 70 & 0 & 70 & 20 \\ 10 & 50 & 0 & 50 & 10 \\ 0 & 20 & 30 & 10 & 0 \end{bmatrix}$$

- Non-edges: $G(x, y) < 25$: Suppressed.

Conclusion

Double Thresholding is an essential step in Canny edge detection, separating strong and weak edges to reduce noise while retaining meaningful edges. The subsequent Edge Tracking by Hysteresis ensures only connected edges are preserved, producing accurate and well-defined edges in the final output.

3.5 Edge Tracking by Hysteresis[8]

Edge Tracking by Hysteresis is the final step in the Canny edge detection algorithm. It determines which weak edges should be preserved based on their connectivity to strong edges, ensuring that only meaningful edges are included in the final output.

Purpose

Double Thresholding in the previous step classifies pixels into strong edges, weak edges, and non-edges:

- Strong edges ($G(x, y) \geq T_{\text{high}}$) are retained.
- Weak edges ($T_{\text{low}} \leq G(x, y) < T_{\text{high}}$) require evaluation.
- Non-edges ($G(x, y) < T_{\text{low}}$) are suppressed.

Edge Tracking by Hysteresis ensures that weak edges connected to strong edges are retained, while isolated weak edges are suppressed.

Process

The process involves the following steps:

1. Start with all strong edge pixels identified from the Double Thresholding step.
2. Traverse the image and examine the 8-connected neighborhood of each strong edge pixel:
   - If a weak edge pixel is connected to a strong edge pixel, mark it as an edge.
   - If a weak edge pixel is not connected to any strong edge, suppress it.
3. Repeat this process for all strong edges until no new weak edges are marked.

Example

Consider a small 5×5 binary image from the Double Thresholding step:

Strong Edges (1), Weak Edges (0.5), Non-Edges (0) :

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0.5 & 0 & 0 \\ 0 & 0.5 & 1 & 0.5 & 0 \\ 0 & 0 & 0.5 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Edge Tracking:

- Starting with strong edges (1), the weak edge (0.5) at position $(3, 4)$ is connected to the strong edge at $(4, 4)$, so it is retained.
- The weak edge at $(2, 3)$ is connected to the strong edge at $(3, 3)$, so it is retained.
- The isolated weak edge at $(1, 3)$ is not connected to any strong edge and is suppressed.

Final result:

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.5 & 0 \\ 0 & 0 & 0.5 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Importance of Hysteresis

Edge Tracking by Hysteresis improves the robustness of the edge detection process by:

- Ensuring that true edges (weak edges connected to strong edges) are not discarded.
- Suppressing spurious edges (isolated weak edges) caused by noise or variations.

Conclusion

Edge Tracking by Hysteresis is a critical step in the Canny edge detection algorithm. It refines the output of Double Thresholding, producing a clean and accurate edge map by retaining meaningful edges while discarding noise and irrelevant features.

3.6 Output of Canny edge detection[9]

The output of the Canny edge detection algorithm is a binary edge map, highlighting the edges in an image while suppressing noise and irrelevant details. This output is a refined result of the sequential processing steps in the algorithm.

Characteristics of the Output

The Canny edge detector produces an image with the following characteristics:

- **Thin Edges**: Due to Non-Maximum Suppression, edges are reduced to one pixel in width, ensuring precise edge localization.
- **Binary Representation**: The edge map consists of:
  - **White Pixels (1)**: Represent detected edges.
  - **Black Pixels (0)**: Represent non-edges.
- **Connected Edges**: Edge Tracking by Hysteresis ensures that weak edges connected to strong edges are preserved, while isolated weak edges are suppressed.
- **Reduced Noise**: Gaussian smoothing minimizes noise, ensuring the edges are not cluttered with irrelevant artifacts.

Example

Consider the following example of an input grayscale image and its corresponding edge map:

Input Image A grayscale image containing an object and some noise:

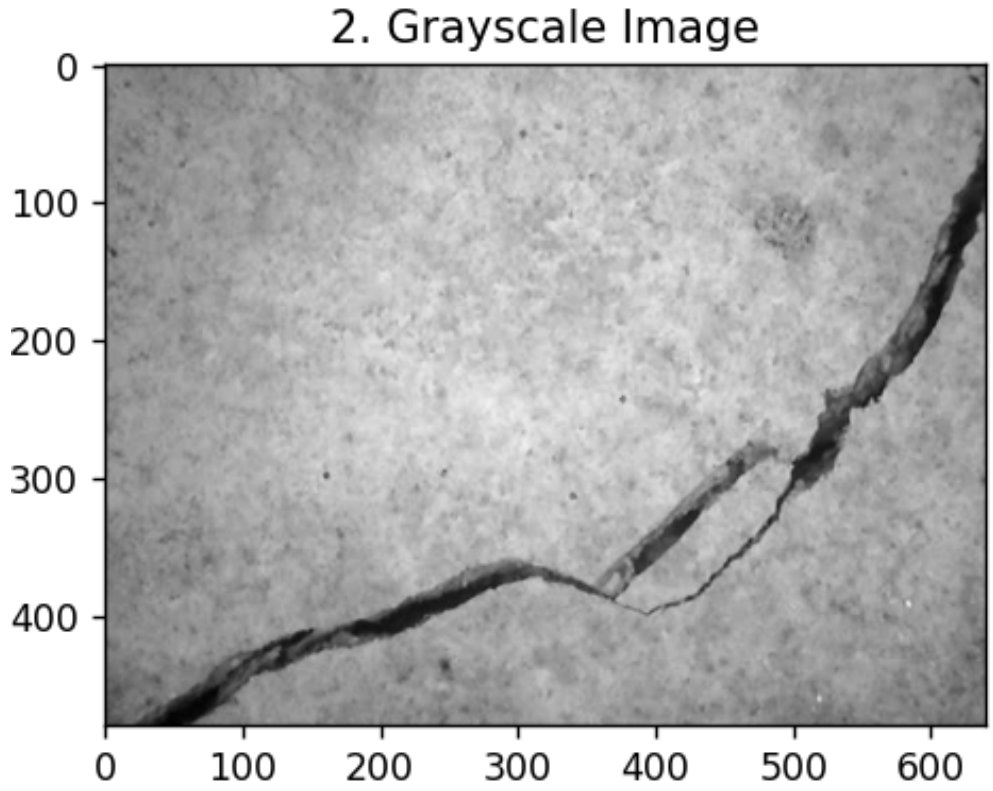


Figure 2: Grayscale image for input

Output Edge Map The binary edge map obtained after applying the Canny edge detection algorithm:

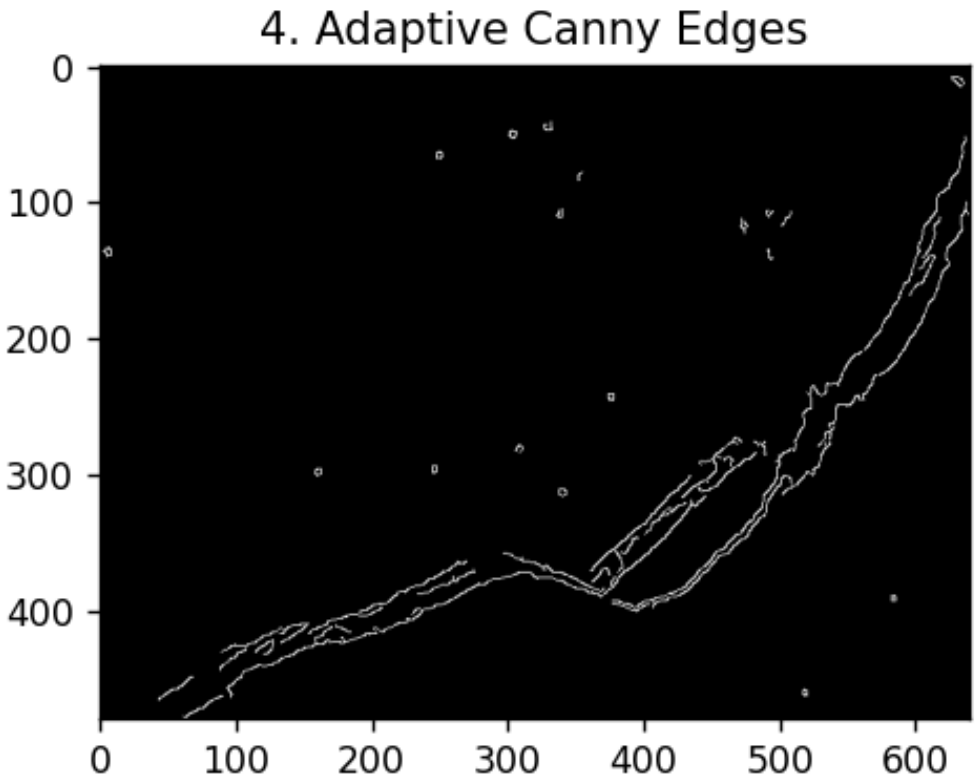


Figure 3: Result image when applying canny

Conclusion

The output of the Canny edge detection algorithm is a clean and accurate binary edge map. It effectively captures the meaningful edges of objects in the image while eliminating noise, making it a powerful tool for a wide range of image processing and computer vision applications.

4 MorphologyEx

This operation refines edges by closing small gaps and then dilating edges to make them more continuous and visible. It consists of two main steps: Dialation and following by Erosion. The purpose of the operation is to fill small holes or gaps in the contours of a binary image. [3]

4.1 Function Syntax

```
cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=2)
```

Syntax explanantion:

- **edges**: The binary input image on which the closing operation is applied.
- **kernel**: The structuring element used for dilation and erosion. The kernel is typically a square matrix, e.g., `np.ones((5, 5), np.uint8)`, which creates a 5×5 array of ones.
- **iterations**: The number of times the image undergoes the morphological transformation.
- **cv2.MORPH_CLOSE**: Specifies the morphological operation to perform. In this case, closing is a dilation followed by an erosion.

4.2 Dilation Operation[3]

Key Components:

- **Sets A and B:**
  - A: The input binary image (foreground is 1s, background is 0s).
  - B: The structuring element (kernel), a smaller binary shape that defines the operation.
- **Translation of B:** $(B)_z$ is the structuring element B, shifted (translated) so that its origin aligns with the pixel z in the image A.
- **Intersection Condition $(B)_z \cap A \neq \emptyset$:** The translated structuring element $(B)_z$ overlaps with A if at least one foreground pixel (1) in B aligns with a foreground pixel (1) in A.

Mathematical Definition of Dilation

The dilation of a set A by a structuring element B, denoted as $A \oplus B$, is defined as:

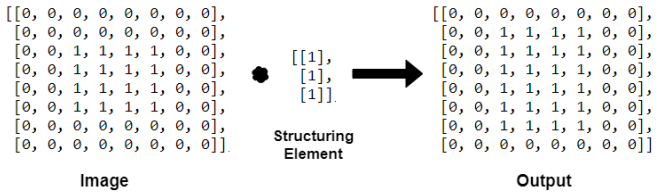
$$A \oplus B = \{z \mid (B)_z \cap A \neq \emptyset\}$$

Explanation of Terms:

- A: The input set, typically representing the foreground pixels in a binary image.
- B: The structuring element (or kernel), defined as a set of coordinates that specifies its shape and size.
- $(B)_z$: The translation of the structuring element B by the point z. Mathematically, this is given by:
  $$(B)_z = \{b + z \mid b \in B\}$$
  Here, every point in B is shifted by the vector z.
- $\cap$: The intersection operator.
- $\neq \emptyset$: Indicates that the intersection is not empty—meaning there is at least one common point between A and the translated structuring element $(B)_z$.

Result of Dilation

The dilation operation "expands" the foreground regions in A. This happens because B only requires partial overlap with A, so boundary pixels are added to the foreground:



4.3 Erosion Operation[3]

Key Components:

- **Sets A and B:**
  - A: The input binary image (foreground is 1s, background is 0s).
  - B: The structuring element (kernel), a smaller binary shape that defines the operation.
- **Translation of B:** $(B)_z$ is the structuring element B, shifted (translated) so that its origin aligns with the pixel z in the image A.
- **Subset Condition $(B)_z \subseteq A$:** The translated structuring element $(B)_z$ is a subset of A if every foreground pixel (1) in B aligns with a foreground pixel (1) in A.

Mathematical Definition of Erosion

The erosion of a set A by a structuring element B, denoted as $A \ominus B$, is defined as:

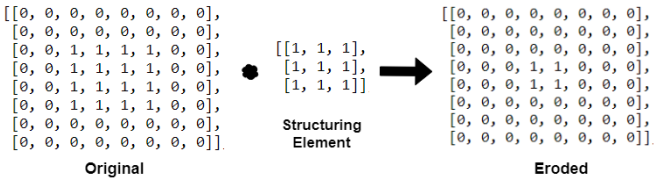
$$A \ominus B = \{z \mid (B)_z \subseteq A\}$$

Explanation of Terms:

- A: The input set, typically representing the foreground pixels in a binary image.
- B: The structuring element (or kernel), defined as a set of coordinates that specifies its shape and size.
- $(B)_z$: The translation of the structuring element B by the point z. Mathematically, this is given by:
  $$(B)_z = \{b + z \mid b \in B\}$$
  Here, every point in B is shifted by the vector z.
- $\subseteq$: Indicates that $(B)_z$ is entirely contained within A.

Result of Erosion

The erosion operation "shrinks" the foreground regions in A. This happens because B must fully fit into A, so boundary pixels in A that cannot fully accommodate B are removed.



5 Contours Detection

The function identifies boundaries of constant intensity regions.Algorithm traces boundary pixels iteratively. It's used in object detection, image segmentation, and shape analysis.[13] Contours are traced using connectivity:[?]

- 4-connectivity: Horizontal/Vertical neighbors.
- 8-connectivity: Includes diagonal neighbors.

5.1 Function syntax[13]

```
cv2.findContours(dilated_edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

Explanation for the code:

- `dilated_edges`: image taken from the previous step
- `cv2.RETR_EXTERNAL`: returning only extreme outer flags
- `cv2.CHAIN_APPROX_SIMPLE`: removes all redundant points and compresses the contour.

The function return 2 values: the contours list and the hierarchy. In the function, the '_' means ignore the hierarchy

5.2 Mathematical Definition[15]

Contours are the boundaries of connected components in a binary image. Mathematically, a contour can be defined as a curve or set of points that satisfy the following condition:

$$C = \{(x, y) \mid I(x, y) = 1 \text{ and } \exists (x', y') \text{ s.t. } I(x', y') = 0\}$$

where $I(x, y)$ represents the pixel intensity at position (x, y). The contour is the set of all boundary points that separate regions with $I(x, y) = 1$ (foreground) from $I(x, y) = 0$ (background). Contours approximate the *level sets* of a function in calculus. If an image $I(x, y)$ is treated as a scalar field, the contours represent the boundary where the field transitions between regions of interest.

5.3 Algorithm of contours detection

1. **Precondition - Binary Image**[16]:
   - `cv2.findContours` operates on binary images, typically obtained by thresholding or edge detection.
   - The input image is a matrix of 0s and 1s, representing the background and foreground, respectively.
2. **Contour Tracing Algorithm**[14]:
   - The function implements variants of algorithms such as *Suzuki's Algorithm* to detect contours.
   - The algorithm scans the binary image row by row and column by column, detecting the start of a new contour when transitioning from a background (0) pixel to a foreground (1) pixel.
   - For every contour:
     - The boundary is traced, identifying a closed loop of pixels.
     - The coordinates of points along the boundary are recorded.

5.4 Results[13]

The result of `cv2.findContours` is:

1. **Contours**: A list of detected contours, where each contour is a NumPy array containing the coordinates of the boundary points. Mathematically:

$$C = \{[(x_1, y_1), (x_2, y_2), \ldots, (x_k, y_k)]\}$$

   Each contour is essentially a polygonal representation of the boundary.

2. **Hierarchy**: A hierarchy describing the relationships between contours (parent-child relationships, outer-inner contours, etc.).

Chapter 5: Final product

1 The source code of crack detection program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image (replace with your image path)
image_path = 'canva.jpg'
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)


#Step 1: Advanced Noise Reduction using Bilateral Filter
# Bilateral filter smooths the image while preserving edges
filtered_img = cv2.bilateralFilter(img, d=3, sigmaColor=75, sigmaSpace=75)

# Step 2: Edge Detection using Adaptive Canny Thresholds
# Compute median of the pixel intensities for adaptive thresholds
median_val = np.median(filtered_img)
lower_thresh = int(max(0, 0.6 * median_val))
upper_thresh = int(min(255, 1.33 * median_val))

# Apply Canny edge detection using the computed adaptive thresholds
edges = cv2.Canny(filtered_img, lower_thresh, upper_thresh)#Detects edges using the Canny edge de

# Step 3: Morphological Operations for better edge continuity
kernel = np.ones((3, 3), np.uint8)

# First, use closing to close small gaps in the cracks
closed_edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=2)

# Then, apply dilation to enhance the cracks further
dilated_edges = cv2.dilate(closed_edges, kernel, iterations=1)

# Step 4: Contour Detection from the dilated edges
contours, _ = cv2.findContours(dilated_edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Filter small contours based on area or length
min_contour_area = 100  # Minimum contour area threshold to remove noise
filtered_contours = [c for c in contours if cv2.contourArea(c) > min_contour_area]

# Step 5: Drawing Contours on the original image for better visualization
output_img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)  # Convert grayscale to BGR for colored drawin
cv2.drawContours(output_img, filtered_contours, -1, (0, 255, 0), 2)  # Draw contours in green

# Visualization using Matplotlib
plt.figure(figsize=(10, 5))
```

```
# Original Image
plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow(img, cmap='gray')

# Final Output with Contours
plt.subplot(1, 3, 3)
plt.title('Crack Detected (Contours)')
plt.imshow(cv2.cvtColor(output_img, cv2.COLOR_BGR2RGB))  # Convert BGR to RGB for correct color d

plt.show()

# Optional: Save the output image with drawn contours
cv2.imwrite('precise_crack_contour_output.jpg', output_img)
```

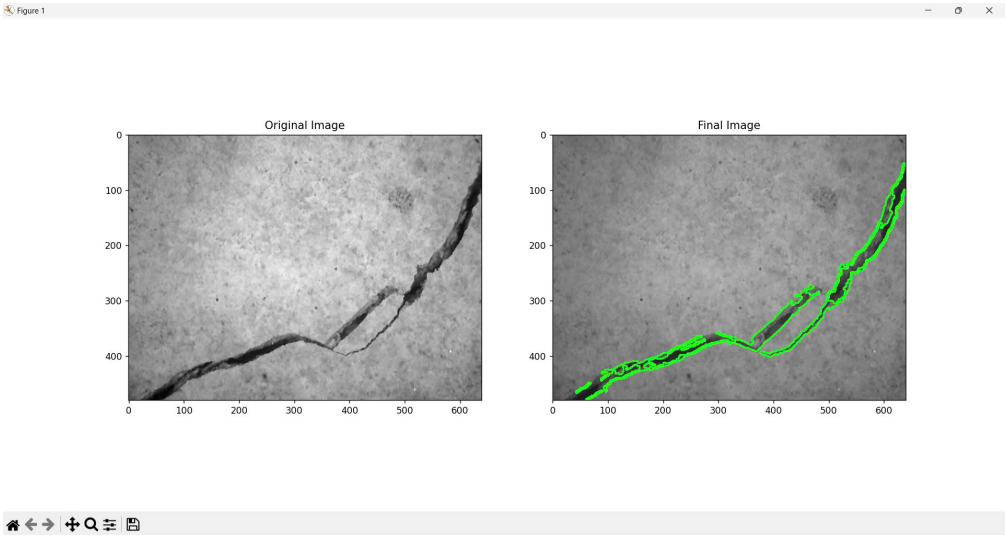


Figure 4: The Final Result

Chapter 6: Conclusion

Conclusion and Recommendations

Through a multi-step process utilizing OpenCV, the crack detection program successfully identifies and highlights cracks, with green lines effectively delineating the external boundaries of the detected cracks in most cases. However, the system exhibits limitations when processing images with dominant colors, such as red or orange, that differ significantly from the typical input dataset. This results in false positives (highlighting non-crack objects) or false negatives (failing to detect actual cracks).

To address these issues and improve the robustness of the detection system:

1. **Color Normalization:** Preprocess the input images to normalize color intensity or convert them into a standardized color space, such as grayscale, to reduce dependency on the original color palette.
2. **Adaptive Thresholding:** Integrate adaptive or context-aware thresholding techniques to dynamically adjust detection parameters based on the image's characteristics.
3. **Advanced Feature Extraction:** Employ machine learning or deep learning models to enhance feature extraction, enabling the program to distinguish cracks from other similar patterns regardless of the image's dominant color.
4. **Dataset Diversification:** Expand the training dataset to include images with varied color profiles and lighting conditions to improve generalization.
5. **Validation Mechanisms:** Implement post-detection validation algorithms to cross-check identified cracks against predefined geometric or textural criteria, minimizing errors.

By addressing these challenges, the program can achieve greater accuracy and reliability across a wider range of input images, making it more suitable for real-world applications.

References

[1] Li, A.R. et al.*Geoenvironmental Disasters* (2024) Geoenvironmental Disasters, SpringerOpen. Available at: https://geoenvironmental-disasters.springeropen.com/ (Accessed: 02 December 2024).

[2] Golewski, G. L *phenomenon* (2023). The phenomenon of cracking in cement concretes and reinforced concrete structures: The mechanism of cracks formation, causes of their initiation, types and places of occurrence, and methods of detection—A review. Buildings, 13(3), 765. Available at: https://www.mdpi.com/1996-1944/13/3/765

[3] Kang, Atul.*Morphological operations*(2019, July 26). Morphological operations in image processing with OpenCV Python. [Blog post]. Available at:https://theailearner.com/tag/morphological-operations/

[4] Arat, M.M. (2020). *RGB to grayscale conversion*, Mustafa Murat ARAT. Available at: https://mmuratarat.github.io/2020-05-13/rgb_to_grayscale_formulas (Accessed: 27 November 2024).

[5] Tutorialspoint, *OpenCV - bilateral filter*. Available at: https://www.tutorialspoint.com/opencv/opencv_bilateral_filter.htm (Accessed: 27 November 2024).

[6] GeeksforGeeks, *Python: Bilateral filtering*. Available at: https://www.geeksforgeeks.org/python-bilateral-filtering/ (Accessed: 27 November 2024).

[7] Ekici, Ouml;zlem *Computer vision*. (2021) Computer vision - Richard Szeliski, Academia.edu. Available at: https://www.academia.edu/48942532/Computer_Vision_Richard_Szeliski(Accessed: 02 December 2024).

[8] OpenCV Documentation (no date) *Canny Edge Detection*. Available at: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

[9] Canny, J. (1986). *A Computational Approach to Edge Detection*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679-698.

[10] Azouz, Z., Hsieh, M. F., & Hamishebahar, S. (2021). A review of computer vision-based crack detection methods in civil infrastructure: Progress and challenges. Sensors, 21(3), 765. Retrieved from https://www.mdpi.com

[11] Intel Corporation. (n.d.). OpenCV: Open Source Computer Vision Library. Retrieved from https://opencv.org

[12] OpenCV. (n.d.). Introduction to OpenCV. Retrieved December 3, 2024, from https://docs.opencv.org/4.x/d1/dfb/intro.html

[13] OpenCV. (n.d.). Contour Features. Retrieved from https://docs.opencv.org/

[14] Sonka, M., Hlavac, V., & Boyle, R. (2014). *Image processing, analysis, and machine vision* (4th ed.). Cengage Learning.

[15] Gonzalez, R. C., & Woods, R. E. (2007). *Digital image processing* (3rd ed.). Pearson Education.

[16] Haralick, R. M., & Shapiro, L. G. (1992). *Computer and robot vision* (Vol. 1). Addison-Wesley.